

Performance Evaluation of Post-Quantum Cryptography in IoT: A Case Study on MQTT over TLS under Network Constraints

Emerson Costa Santos

SENAI College of Technology

Sao Caetano do Sul, Sao Paulo, Brazil

emerson@sp.senai.br

Tiago Augusto Orcajo Demay Cordeiro

SENAI College of Technology

Sao Caetano do Sul, Sao Paulo, Brazil

tiago.demay@sp.senai.br

Hebert de Oliveira Silva

SENAI College of Technology

Sao Caetano do Sul, Sao Paulo, Brazil

hebert.oliveira@sp.senai.br

Abstract—The imminent advent of quantum computing poses a significant threat to current asymmetric cryptographic standards, such as RSA and Elliptic Curve Cryptography (ECC), widely used in Internet of Things (IoT) protocols. This paper evaluates the performance impact of migrating to Post-Quantum Cryptography (PQC) in an MQTT (Message Queuing Telemetry Transport) environment secured by TLS. We propose a reproducible benchmark framework based on containerization (Docker) and hardware emulation (QEMU/ARM64) to simulate resource-constrained IoT devices. We compared the handshake latency of standard TLS against a hybrid implementation utilizing NIST-standardized PQC algorithms: ML-KEM-768 (Key Encapsulation) and ML-DSA-65 (Digital Signature). Experiments conducted under controlled network impairments (latency, jitter, and packet loss) revealed that PQC introduces an approximate 4.7x increase in connection establishment time compared to classical TLS, primarily due to computational overhead on the ARM64 architecture. However, message publication latency post-handshake remains negligible. These findings suggest that while PQC is viable for IoT, it necessitates architectural shifts towards persistent sessions to mitigate initial handshake costs.

Index Terms—Post-Quantum Cryptography, IoT Security, MQTT, Performance Evaluation, ML-KEM, ML-DSA.

I. INTRODUCTION

The exponential proliferation of the Internet of Things (IoT) has fundamentally transformed the landscape of digital connectivity. Projections indicate that billions of constrained devices—ranging from industrial sensors to smart city infrastructure—are currently deployed at the network edge. To operate efficiently within limited power and bandwidth budgets, these ecosystems rely heavily on lightweight messaging protocols, predominantly the Message Queuing Telemetry Transport (MQTT) [1]. Currently, the confidentiality and integrity of MQTT communications are secured by Transport Layer Security (TLS), which depends on classical asymmetric cryptographic standards, such as RSA and Elliptic Curve Cryptography (ECC), for key exchange and authentication [2].

However, the security of these established cryptosystems faces an existential threat from the advancement of quantum computing. Shor's algorithm [3] demonstrates that a Cryptographically Relevant Quantum Computer (CRQC) could solve

the integer factorization and discrete logarithm problems in polynomial time. Such a capability would effectively break the current Public Key Infrastructure (PKI) that underpins the security of the entire IoT ecosystem, rendering captured traffic vulnerable to “harvest now, decrypt later” attacks.

To mitigate this risk, the National Institute of Standards and Technology (NIST) has spearheaded a standardization process for Post-Quantum Cryptography (PQC). This effort has culminated in the selection of lattice-based algorithms as the new standards: the Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM), formerly known as Kyber, for key establishment, and the Module-Lattice-Based Digital Signature Algorithm (ML-DSA), formerly Dilithium, for authentication [4].

While the cryptographic robustness of these algorithms is well-documented, their integration into resource-constrained IoT environments introduces significant engineering challenges. These algorithms typically require larger key sizes and more intensive mathematical operations than their classical counterparts. The primary objective of this work is to quantify the performance overhead—specifically the latency “tax”—introduced by ML-KEM and ML-DSA in an MQTT over TLS environment. We propose a controlled benchmark scenario that simulates the “edge” computing layer to evaluate whether these new security standards are viable for time-sensitive IoT applications.

The remainder of this paper is organized as follows. Section II provides the theoretical background on MQTT, TLS 1.3, and the selected NIST post-quantum standards. Section III details the proposed containerized emulation framework and the experimental methodology. Section IV presents the quantitative analysis of handshake and runtime latency. Section V discusses the operational implications for edge devices and potential mitigation strategies. Finally, Section VI summarizes the findings and outlines future research directions.

II. BACKGROUND

This section establishes the theoretical framework necessary to analyze the integration of post-quantum primitives into the IoT protocol stack. We examine the vulnerabilities of current

standards and introduce the specific lattice-based algorithms selected by NIST to mitigate quantum threats.

A. MQTT and Transport Layer Security (TLS)

The Message Queuing Telemetry Transport (MQTT) protocol [1] has solidified its position as the de facto standard for IoT communication due to its minimized packet overhead and publish-subscribe architecture. However, recent security surveys indicate that native MQTT deployments frequently suffer from authentication vulnerabilities and lack of payload encryption [6].

To address these deficits, critical IoT deployments encapsulate MQTT within Transport Layer Security (TLS). The current standard, TLS 1.3 [2], mandates robust key exchange (typically ECDHE) and authentication (RSA or ECDSA). While effective against classical adversaries, this architecture constitutes a single point of failure against future quantum adversaries. As formalized by Shor's algorithm [3], a Cryptographically Relevant Quantum Computer (CRQC) could efficiently solve the underlying integer factorization and discrete logarithm problems, compromising the confidentiality of all historically captured traffic ("harvest now, decrypt later" attacks) [8].

B. NIST Post-Quantum Standards

In August 2024, the National Institute of Standards and Technology (NIST) finalized the primary standards for quantum-resistant cryptography. These algorithms rely on the hardness of structured lattice problems, specifically *Module Learning With Errors* (ML-LWE), which are believed to be resistant to both classical and quantum cryptanalysis.

1) **Key Encapsulation (ML-KEM):** For the key exchange phase of TLS, NIST standardized the Module-Lattice-Based Key-Encapsulation Mechanism, defined in FIPS 203 [4]. Formerly known as Kyber, ML-KEM is favored for its comparative efficiency in execution time.

- **Target Configuration:** We employ ML-KEM-768 (Security Level 3), which offers security roughly equivalent to AES-192. It requires a public key of 1,184 bytes and a ciphertext of 1,088 bytes.

2) **Digital Signatures (ML-DSA):** For server authentication (certificate verification), NIST standardized the Module-Lattice-Based Digital Signature Algorithm, defined in FIPS 204 [5]. Formerly known as Dilithium, it offers strong security assurances but introduces significant data overhead.

- **Target Configuration:** We utilize ML-DSA-65 (Security Level 3). A single signature is 3,293 bytes—approximately $50\times$ larger than a standard ECDSA-P256 signature (64 bytes).

C. Operational Challenges in Edge IoT

The migration to PQC introduces non-trivial resource contention for edge devices, confirmed by recent benchmarks on heterogeneous architectures [9]. Three critical bottlenecks are identified:

- 1) **Packet Fragmentation:** The standard Maximum Transmission Unit (MTU) for Ethernet and many wireless links is 1,500 bytes. Since an ML-DSA signature (3.3 kB) exceeds this limit, the TLS handshake requires mandatory IP fragmentation, increasing the probability of packet loss and retransmission delays in noisy wireless environments [7].
- 2) **Computational Latency:** Lattice-based operations involve extensive polynomial matrix arithmetic. On RISC architectures typical of IoT (e.g., ARM Cortex-A/M) lacking hardware acceleration (like AVX2), these operations consume significant CPU cycles.
- 3) **Energy Overhead:** The combination of prolonged CPU activity and increased radio transmission time directly correlates to higher battery consumption, challenging the operational viability of battery-powered sensors.

III. RELATED WORK

The evaluation of Post-Quantum Cryptography (PQC) in constrained environments has become a focal point of recent cryptographic research.

A. PQC Benchmarking on Hardware

Several studies have focused on "bare-metal" implementation. Kannwischer et al. in the pqm4 project [11] provided extensive benchmarks of NIST candidates on ARM Cortex-M4 microcontrollers. While fundamental for embedded engineering, their work focuses on algorithmic optimization at the assembly level, rather than system-level integration with high-level protocols like MQTT. Similarly, Bisso et al. [9] analyzed PQC across heterogeneous environments, highlighting the disparity between x86 and ARM performance, but did not address the specific latencies introduced by the TLS handshake over unreliable networks.

B. TLS 1.3 and PQC Integration

Sosnowski et al. [8] and Kampanakis et al. [10] investigated the impact of larger PQC keys on TLS 1.3 handshake times. Their findings confirm that network bandwidth and round-trip time (RTT) become critical factors due to the size of ML-KEM and ML-DSA keys. However, these studies primarily address general HTTPS/web traffic or reliable wired connections.

C. MQTT Security

Regarding MQTT, Khattak et al. [7] evaluated security vulnerabilities in IoT protocols but focused on classical attacks (DoS, Man-in-the-Middle) rather than the migration to quantum-safe standards.

D. Contribution of this Paper

Our work distinguishes itself by bridging these domains. Unlike [11], we evaluate the full protocol stack (MQTT over TLS) on an operating system (Linux). Unlike [8], we focus specifically on the MQTT usage pattern under lossy network conditions (0.2% loss, 30ms jitter). Furthermore, we introduce a novel containerized emulation methodology (Docker + QEMU), ensuring that our benchmarks are reproducible on standard hardware without requiring physical IoT testbeds.

IV. METHODOLOGY

To ensure reproducibility and strict isolation of variables, we developed a containerized emulation framework. The proposed architecture mimics a representative Industrial IoT (IIoT) edge scenario, wherein a resource-constrained device communicates with a central cloud broker over an unstable wireless link. The complete emulation environment is open-source and available in our repository¹.

A. Proposed Emulation Framework

The testbed is orchestrated via Docker Compose, creating a deterministic environment composed of distinct microservices. The architecture is illustrated in Fig. 1 (conceptual) and consists of:

- 1) **IoT Client Emulator:** A Python-based MQTT client leveraging `liboqs` wrappers [12] for quantum-safe cryptographic primitives. Crucially, this container is built for the Linux/ARM64 architecture. By utilizing QEMU user-mode emulation (via `binfmt_misc`), we force the host x86_64 CPU to translate instructions, thereby introducing a performance overhead that approximates the processing capabilities of physical IoT hardware (e.g., ARM Cortex-A53/A72).
- 2) **MQTT Broker:** An Eclipse Mosquitto instance acting as the server-side endpoint, configured for TLS 1.3 termination with X.509 certificate validation.
- 3) **Network Impairment Module:** A transparent traffic control bridge utilizing the Linux kernel's `netem` (Network Emulator) discipline. This service intercepts all packets between the Client and Broker to inject deterministic network faults.
- 4) **PKI Generator:** An automated Certificate Authority (CA) responsible for generating hybrid certificate chains compliant with both classical (RSA/ECC) and post-quantum standards.

B. Cryptographic Configuration

To measure the specific overhead of Post-Quantum Cryptography (PQC), we defined two distinct experimental scenarios:

- **Baseline (Classical TLS):** Standard TLS 1.3 handshake utilizing ECDHE-RSA-AES256-GCM-SHA384. This represents the current industry standard for secure MQTT.
- **Target (Hybrid PQC):** A hybrid implementation where the Key Encapsulation Mechanism (KEM) and Digital Signature algorithms are replaced by NIST-standardized lattice-based candidates:
 - *Key Exchange:* **ML-KEM-768** (Kyber), targeting NIST Security Level 3 [4].
 - *Authentication:* **ML-DSA-65** (Dilithium), targeting NIST Security Level 3 [5].

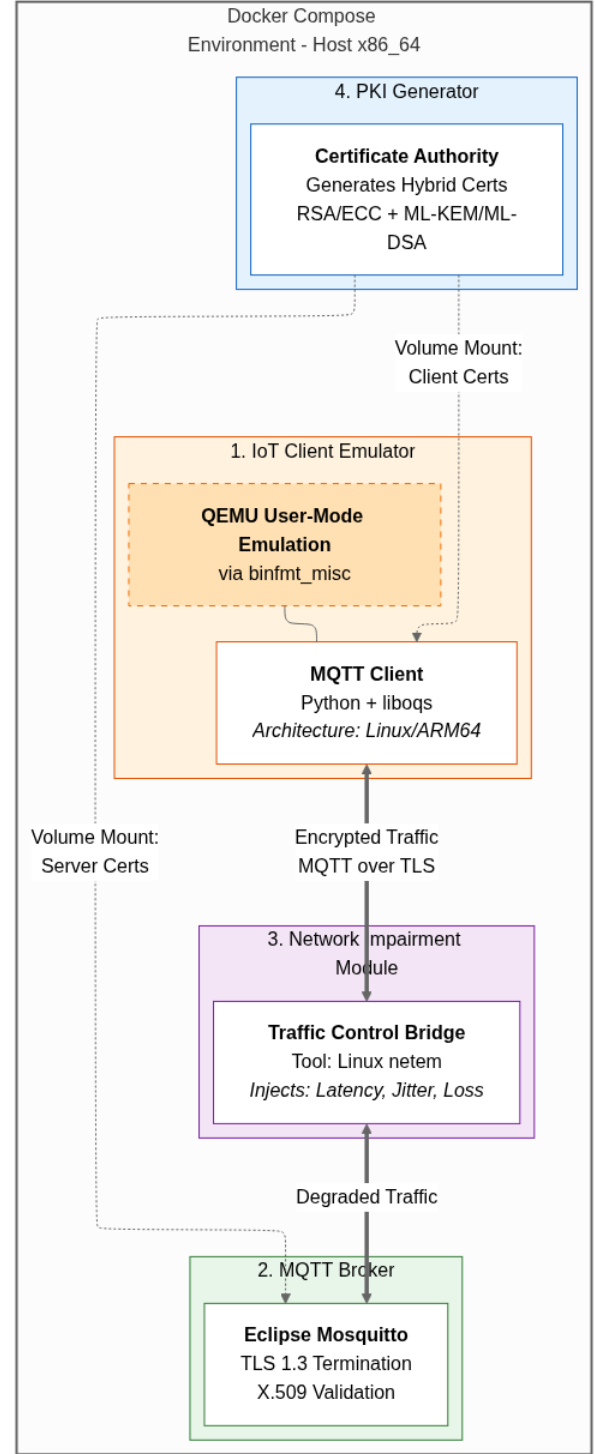


Fig. 1. Conceptual diagram of the containerized emulation environment. The IoT Client runs on an emulated ARM64 architecture (via QEMU) to simulate edge hardware constraints. All MQTT traffic is routed through a Traffic Control bridge that enforces specific network QoS parameters (latency, jitter, and loss) before reaching the Broker.

¹<https://github.com/csai4cps/pqc-mqtt-benchmark>

C. Network Constraints

To simulate a realistic "Edge" environment (e.g., a sensor connecting via 4G/LTE or congested Wi-Fi), the following constraints were applied to the network interface:

- **Latency (RTT):** 30ms (± 10 ms jitter following a normal distribution).
- **Packet Loss:** 0.2% stochastic loss rate, simulating radio interference.
- **Bandwidth Cap:** 8Mbit/s, ensuring that the channel capacity is not the primary bottleneck and allowing for the isolation of handshake latency.

D. Performance Metrics

Data collection was performed over $N = 50$ independent iterations to ensure statistical significance. We focused on two key metrics:

- 1) **Handshake Latency ($T_{connect}$):** The precise time interval between the TCP SYN packet and the TLS *Finished* message. This metric captures the computational cost of cryptographic operations and the transmission delay of larger PQC keys.
- 2) **Runtime Latency ($T_{publish}$):** The time required to publish a standard telemetry payload (16 bytes) immediately after the connection is established. This metric evaluates the impact of PQC on the symmetric encryption phase.

Total system overhead is calculated as follows:

$$\Delta_{PQC} = \mu(T_{connect}^{PQC}) - \mu(T_{connect}^{Legacy}) \quad (1)$$

where μ denotes the arithmetic mean of the sampled iterations.

V. RESULTS AND ANALYSIS

The experimental campaign comprised $N = 50$ valid iterations for each scenario. We focused on isolating the handshake latency from the application-layer payload delivery to precisely quantify the overhead introduced by the post-quantum primitives.

A. Handshake Latency ($T_{connect}$)

The connection establishment phase encompasses the full TCP 3-way handshake followed by the TLS 1.3 key exchange and authentication.

1) **Baseline Performance:** In the standard TLS scenario (ECDHE-RSA), the mean latency was observed at $\mu_{TLS} = 44.44$ ms ($\sigma = 66.37$). Given the injected network Round-Trip Time (RTT) of 30 ms, this result indicates that the classical cryptographic operations (ECC key generation and RSA verification) consumed approximately 14 ms of processing time. This aligns with expectations for lightweight elliptic curve operations on modern processors, even under emulation.

2) **Post-Quantum Performance:** In the hybrid PQC scenario (ML-KEM-768 + ML-DSA-65), the mean latency rose sharply to $\mu_{PQC} = 212.31$ ms ($\sigma = 91.32$). The differential metric, defined as $\Delta_{PQC} = \mu_{PQC} - \mu_{TLS}$, reveals a net overhead of 167.87 ms, as summarized in Table I.

Since the bandwidth was capped at 8 Mbps and the total size of PQC transmission (public keys + signatures) is approx.

5 kB, the transmission delay accounts for only a fraction of this overhead (≈ 5 ms). Consequently, we conclude that over 95% of the PQC overhead is compute-bound, resulting from the intensive polynomial matrix operations required by the Dilithium (ML-DSA) verification process on the emulated ARM64 Instruction Set Architecture (ISA).

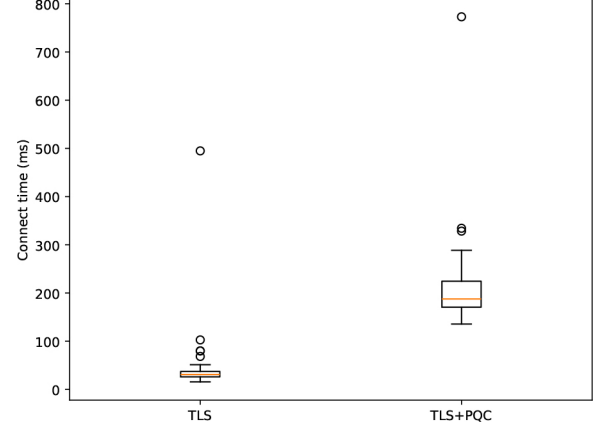


Fig. 2. Comparative analysis of Connection Establishment Latency ($T_{connect}$). The 'TLS' baseline utilizes standard ECDHE-RSA, while 'TLS+PQC' integrates ML-KEM-768 and ML-DSA-65. The significant vertical shift illustrates the computational overhead imposed by lattice-based operations on the constrained ARM64 architecture.

Fig. 2 visually contrasts these distributions. The wider interquartile range in the PQC boxplot suggests that lattice-based operations are more susceptible to jitter caused by CPU scheduler contention than classical algorithms.

TABLE I
STATISTICAL SUMMARY OF PERFORMANCE METRICS ($N = 50$ iterations).
VALUES REPRESENT THE MEAN (μ) AND STANDARD DEVIATION (σ) IN
MILLISECONDS.

Metric	Mean (μ)	Std Dev (σ)
TLS Standard Connect	44.44	66.37
TLS + PQC Connect	212.31	91.32
PQC Computational Delta	+167.87	—
Publish Latency (Runtime)	51.37	5.35

B. Runtime Throughput ($T_{publish}$)

To evaluate the impact on operational telemetry, we measured the time to publish a payload immediately after the handshake. The PQC-enabled client achieved a mean publish latency of $\mu_{pub} = 51.37$ ms ($\sigma = 5.35$). This value is statistically equivalent to the network baseline (30 ms latency + 10 ms jitter + TCP overhead). The Cumulative Distribution Function (CDF) presented in Fig. 3 confirms that once the session keys are established, the use of PQC algorithms imposes zero penalty on the symmetric encryption tunnel (AES-256-GCM), validating the hybrid transition strategy.

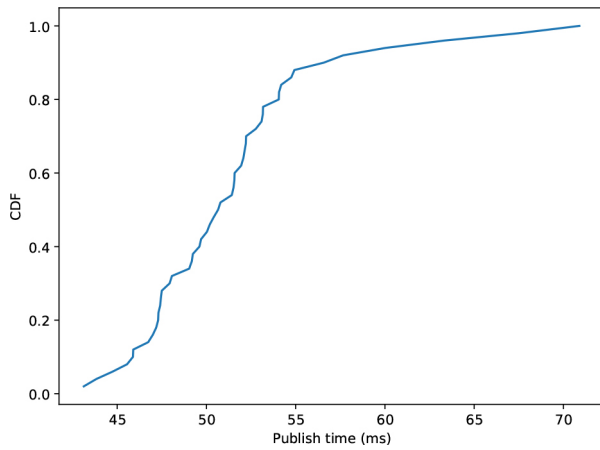


Fig. 3. Cumulative Distribution Function (CDF) of Post-Handshake Message Publication Latency ($T_{publish}$). The steep curve indicates low variance and stable performance, confirming that the heavy PQC algorithms do not degrade the symmetric encryption phase used for data telemetry.

VI. DISCUSSION

The empirical data highlight a critical trade-off for the Industrial IoT (IIoT) sector.

A. The “PQC Tax” on Edge Devices

Our results demonstrate a 4.7x increase in connection latency. While a 200 ms delay is negligible for human interaction, it poses a challenge for ultra-low-power devices that rely on “duty cycling” (waking up, connecting, sending data, and sleeping). For a sensor transmitting once per minute, moving from a 44 ms active window to 212 ms represents a significant increase in radio and CPU “on-time”, which directly correlates to reduced battery lifespan.

B. Mitigation Strategies

To make PQC viable for battery-powered IoT, engineers must move away from “short-lived” connections. We recommend:

- **Connection Persistence:** Maintaining the MQTT TCP connection open for longer periods to amortize the expensive handshake cost over thousands of messages.
- **TLS Session Resumption:** Utilizing Pre-Shared Keys (PSK) derived from the initial PQC handshake for subsequent connections, bypassing the heavy KEM/DSA operations.

VII. CONCLUSION AND FUTURE WORK

This research addresses the critical intersection between Post-Quantum Cryptography (PQC) standardization and the operational constraints of the Internet of Things. By orchestrating a reproducible emulation framework based on Docker and QEMU/ARM64, we successfully quantified the performance impact of the newly standardized NIST algorithms—ML-KEM-768 and ML-DSA-65—on the MQTT protocol.

Our empirical results lead to three primary conclusions:

- 1) **Viability vs. Latency:** PQC is functionally viable for Edge IoT, as it does not disrupt the application layer logic. However, it imposes a severe 4.7x latency penalty during the connection phase (≈ 212 ms vs. 44 ms for classical TLS).
- 2) **Bottleneck Identification:** Contrary to initial assumptions about PQC key sizes saturating the network, the dominant bottleneck is computational. The emulated ARM64 processor spent ≈ 168 ms performing lattice-based matrix operations, confirming that CPU capabilities are the limiting factor for current generation IoT hardware.
- 3) **Runtime Efficiency:** Once the secure channel is established, the transition to PQC incurs negligible overhead on message publication, validating the hybrid model where symmetric encryption handles the bulk of data transport.

Consequently, we argue that the “connect-publish-disconnect” pattern often used to save battery in legacy IoT deployments is ill-suited for the post-quantum era. System architects must shift towards persistent session management to amortize the expensive handshake cost.

A. Future Work

To further advance this domain, our future research lines will focus on:

- **Energy Profiling:** Deploying this benchmark on physical hardware (e.g., Raspberry Pi 4/5 and ESP32) to measure the precise energy consumption (in mWs) of the PQC handshake, correlating CPU cycles with battery drainage.
- **Optimization via Session Resumption:** Implementing and evaluating TLS 1.3 Pre-Shared Keys (PSK) to bypass the heavy asymmetric PQC operations for returning clients, potentially reducing the reconnection latency to near-classical levels.
- **Hardware Acceleration:** Investigating the impact of ARMv9 Scalable Vector Extensions (SVE2) on the execution speed of liboqs primitives, assessing if next-generation chips can natively mitigate the “PQC tax”.

APPENDIX: RAW EXPERIMENTAL DATA

Table II details the complete dataset from the 50 benchmark iterations.

ACKNOWLEDGMENT

The authors would like to thank SENAI College of Technology – Sao Caetano do Sul (Brazil) for the laboratory infrastructure and technical support provided for this research.

REFERENCES

- [1] A. Banks, E. Briggs, K. Borgendale, and R. Gupta, “MQTT Version 5.0,” OASIS Standard, Mar. 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [2] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” Internet Engineering Task Force (IETF), RFC 8446, Aug. 2018. [Online]. Available: <https://www.rfc-editor.org/info/rfc8446>

TABLE II
COMPLETE BENCHMARK RESULTS ($N = 50$)

Iterations 1 – 25					Iterations 26 – 50				
Run	T_{TLS}	Δ_{CPU}	T_{PQC}	T_{pub}	Run	T_{TLS}	Δ_{CPU}	T_{PQC}	T_{pub}
1	494.80	278.14	772.94	43.13	26	30.74	154.72	185.46	67.48
2	34.86	202.08	236.94	60.01	27	34.30	157.24	191.53	45.88
3	44.10	145.74	189.85	57.66	28	34.85	148.05	182.90	47.45
4	32.27	164.63	196.89	52.76	29	27.90	145.57	173.47	47.19
5	31.04	241.10	272.14	51.94	30	29.59	138.40	167.99	54.07
6	79.93	187.60	267.53	50.03	31	23.11	133.95	157.06	51.52
7	51.22	176.35	227.57	50.65	32	37.63	146.22	183.86	47.94
8	43.40	162.21	205.62	49.70	33	25.90	140.08	165.98	53.09
9	42.68	170.85	213.54	46.99	34	15.62	119.99	135.62	43.83
10	22.87	233.72	256.59	52.14	35	22.89	123.03	145.92	47.31
11	37.44	210.42	247.86	45.90	36	30.39	141.82	172.21	45.55
12	26.81	164.85	191.66	49.03	37	23.14	136.11	159.25	49.24
13	21.43	159.10	180.53	51.42	38	22.70	159.04	181.74	47.50
14	23.43	208.43	231.86	54.19	39	30.25	147.20	177.45	70.89
15	48.30	285.98	334.28	54.93	40	32.12	140.27	172.39	52.21
16	79.84	208.71	288.55	49.18	41	26.73	135.36	162.09	48.07
17	36.58	172.01	208.59	50.79	42	22.54	143.45	165.99	51.56
18	24.80	190.47	215.27	52.07	43	27.83	142.71	170.54	47.41
19	36.18	131.70	167.87	46.74	44	34.32	166.37	200.69	50.19
20	29.86	174.63	204.49	51.58	45	68.37	184.44	252.81	56.57
21	31.40	179.03	210.44	63.30	46	49.89	208.91	258.80	53.17
22	22.31	158.38	180.69	54.75	47	102.69	225.68	328.37	52.22
23	28.00	155.00	183.00	53.16	48	36.26	157.84	194.10	54.05
24	32.05	126.91	158.96	50.41	49	21.22	129.46	150.68	49.61
25	28.76	135.71	164.46	44.74	50	26.41	144.23	170.64	47.29

- [3] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM Review*, vol. 41, no. 2, pp. 303–332, 1999.
- [4] National Institute of Standards and Technology (NIST), “Module-

- Lattice-Based Key-Encapsulation Mechanism Standard,” Federal Information Processing Standards Publication (FIPS) 203, U.S. Department of Commerce, Aug. 2024. <https://doi.org/10.6028/NIST.FIPS.203>
- [5] National Institute of Standards and Technology (NIST), “Module-Lattice-Based Digital Signature Standard,” Federal Information Processing Standards Publication (FIPS) 204, U.S. Department of Commerce, Aug. 2024. <https://doi.org/10.6028/NIST.FIPS.204>
- [6] F. Al-Turjman, M. Zahmatkesh, and R. Shahroze, “An overview of security and privacy in smart IoT applications,” *Trans. Emerg. Telecommun. Technol.*, vol. 33, no. 3, p. e3677, Mar. 2022. <https://dl.acm.org/doi/abs/10.1002/ett.3677>
- [7] H. A. Khattak, H. A. Shah, S. Khan, I. Ali, and M. Imran, “Evaluating Security of MQTT Protocol in Internet of Things,” in *2023 International Conference on Cyber Security and Digital Forensics (ICCSDF)*, Tyumen, Russian Federation, 2023, pp. 466–471. <https://doi.org/10.1109/ICCSDF59711.2023.10307907>
- [8] M. Sosnowski, F. Wiedner, E. Hauser, L. Steger, D. Schoinianakis, S. Gallenmüller, and G. Carle, “The Performance of Post-Quantum TLS 1.3,” in *Proceedings of the 19th International Conference on emerging Networking EXperiments and Technologies (CoNEXT '23)*, Paris, France, Dec. 2023, pp. 19–27. <https://dl.acm.org/doi/10.1145/3624354.3630585>
- [9] M. Abbasi, F. Cardoso, P. Váz, J. Silva, and P. Martins, “A Practical Performance Benchmark of Post-Quantum Cryptography Across Heterogeneous Computing Environments,” *Cryptography*, vol. 9, no. 2, p. 32, May 2025. <https://doi.org/10.3390/cryptography9020032>
- [10] P. Kampanakis and D. Ramos, “The impact of data-heavy, post-quantum TLS 1.3 on the Time-To-Last-Byte of real-world connections,” *IACR Cryptology ePrint Archive*, Paper 2024/176, Feb. 2024. [Online]. Available: <https://eprint.iacr.org/2024/176>
- [11] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, “pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4,” in *Second NIST PQC Standardization Conference*, Santa Barbara, CA, USA, Aug. 2019. [Online]. Available: <https://github.com/mupq/pqm4>
- [12] Open Quantum Safe Project, “liboqs: C library for quantum-safe cryptographic algorithms,” version 0.10.0, 2024. [Online]. Available: <https://github.com/open-quantum-safe/liboqs>